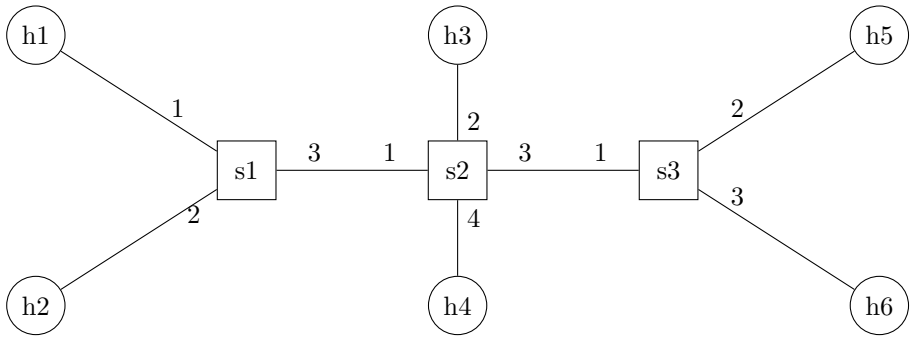
Piattaforme Software per la Rete (Mod. 1) – 12/09/2016

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Il controllore Ryu è collegato ai nodi s1,..., sm ed è configurato come indicato nel Programma 1. Le interfacce tra gli switch e il controllore non sono indicate in figura.



(a) Scrivere il contenuto delle tabelle openflow dopo l'accensione di tutti i nodi.

Soluzione:

s1

Priority	Match	Action	pkt cnt
0	*	output controller	0

s2

Priority	Match	Action	pkt cnt
0	*	output controller	0
2	eth_type = 0x800 ip_proto = 17 udp_dst = 123		

s3

Priority	Match	Action	pkt cnt
0	*	output controller	0

L'host h1, porta 3456 invia un datagramma UDP a h5, porta 123.

(b) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s1, s2, s3.

Soluzione:

$s1 \rightarrow C$: packet_in(MAC1 > MAC5 IPv4 IP1 > IP5 UDP 3456 > 123)

$s1 \rightarrow s2$,$h2$: MAC1 > MAC5 IPv4 IP1 > IP5 UDP 3456 > 123

s2 scarta il pacchetto e s3 non lo riceve

L'host h2, porta 5432 invia una richiesta di apertura di connessione TCP a h5, porta 80.

(c) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

Soluzione:
$s2 \rightarrow C$: MAC2,MAC5,IP2,IP5,TCP,5432,80 [SYN]
$s2 \rightarrow \{s3,h3,h4\}$: MAC2,MAC5,IP2,IP5,TCP,5432,80 [SYN]

Il server risponde.

(d) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

Soluzione:
$s2 \rightarrow C$: MAC5,MAC2,IP5,IP2,TCP,80,5432 [SYN ACK]
$s2 \rightarrow \{s1,h3,h4\}$: MAC2,MAC5,IP5,IP2,TCP,80,5432 [SYN ACK]

L'host h3, porta 5432 invia un datagramma UDP a h2, porta 80.

(e) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

Soluzione:
$s2 \rightarrow C$: MAC3,MAC2,IP3,IP2,UDP,5432,80
$s2 \rightarrow \{s1,s3,h4\}$: MAC3,MAC2,IP3,IP2,UDP,5432,80

L'host h3, porta 5432 invia un altro datagramma UDP a h2, porta 80.

(f) Scrivere tutti i pacchetti uscenti da tutte le interfacce di s2.

Soluzione:
$s2 \rightarrow \{s1,s3,h4\}$: MAC3,MAC2,IP3,IP2,UDP,5432,8080

(g) Scrivere il contenuto finale delle tabelle openflow.

Soluzione:

s1

Priority	Match	Action	pkt cnt
0	*	output controller	
3	eth_type = 0x800 ip_src = IP2 ip_dst = IP5 ip_proto = 6	output(flood)	

s2

Priority	Match	Action	pkt cnt
0	*	output controller	
2	eth_type = 0x800 ip_proto = 17 udp_dst = 123		
3	eth_type = 0x800 ip_src = IP2 ip_dst = IP5 ip_proto = 6	output(flood)	
3	eth_type = 0x800 ip_src = IP2 ip_dst = IP5 ip_proto = 17	output(flood) udp_dst=8080 output(flood)	

s3

Priority	Match	Action	pkt cnt
0	*	output controller	
3	eth_type = 0x800 ip_src = IP2 ip_dst = IP5 ip_proto = 6	output(flood)	
3	eth_type = 0x800 ip_src = IP2 ip_dst = IP5 ip_proto = 17	output(flood) udp_dst=8080 output(flood)	

Programma 1 (1/2) – Switch Features

```
@set_ev_cls(ofp_event.EventOFPSwitchFeatures, CONFIG_DISPATCHER)
def switch_features_handler(self, ev):
    datapath = ev.msg.datapath
    ofproto = datapath.ofproto
    parser = datapath.ofproto_parser

    match = parser.OFPMatch()
    actions = [parser.OFPActionOutput(ofproto.OFPP_CONTROLLER,
                                      ofproto.OFPCML_NO_BUFFER)]
    inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
    mod = parser.OFPFlowMod(datapath=datapath, priority=0,
                            match=match, instructions=inst)
    datapath.send_msg(mod)

    if (datapath.id == 2):
        match = parser.OFPMatch(
            eth_type=0x0800,     # IPv4
         ip_proto=17,        # UDP
            udp_dst=123)
        actions = [  ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=2,
                                match=match, instructions=inst)
        datapath.send_msg(mod)
```

Programma 1 (2/2) – Packet In

```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
    msg = ev.msg; datapath = msg.datapath
    ofproto = datapath.ofproto; parser = datapath.ofproto_parser
    in_port = msg.match['in_port']
    pkt = packet.Packet(msg.data)
    pkt_eth = pkt.get_protocol(eth.eth)
    pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
    pkt_udp = pkt.get_protocol(udp.udp)
    pkt_tcp = pkt.get_protocol(tcp.tcp)

    if (pkt_tcp is not None) and (pkt_tcp.dst==80):
        match = parser.OFPMatch(
            eth_type=0x0800,     # IPv4
            ip_src=pkt_ipv4.src,
            ip_dst=pkt_ipv4.dst,
            ip_proto=6)          # TCP
        actions = [ parser.OFPActionOutput(ofproto.OFPP_FLOOD) ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=3,
                                match=match, instructions=inst)
        datapath.send_msg(mod)

    if (pkt_udp is not None) and (pkt_udp.dst==80):
        match = parser.OFPMatch(
            eth_type=0x0800,     # IPv4
            ip_src=pkt_ipv4.src,
            ip_dst=pkt_ipv4.dst,
            ip_proto=17)          # TCP
        actions = [
            parser.OFPActionSetField(udp_dst=8080)
            parser.OFPActionOutput(ofproto.OFPP_FLOOD)
        ]
        inst = [parser.OFPInstructionActions(ofproto.OFPIT_APPLY_ACTIONS,actions)]
        mod = parser.OFPFlowMod(datapath=datapath, priority=3,
                                match=match, instructions=inst)
        datapath.send_msg(mod)

    data = msg.data
    actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
    out = parser.OFPPacketOut(datapath=datapath, buffer_id=msg.buffer_id,
                              in_port=in_port, actions=actions, data=data)
    datapath.send_msg(out)
```

Esercizio 2

Si consideri un sistema di misura del traffico in un router. Ai pacchetti entranti da tutti i link è associato un identificativo di flusso x ottenuto concatenando alcuni campi del pacchetto. La lunghezza massima dei pacchetti è 1024 byte.

Sull'identificativo x sono calcolati n hash usando la famiglia di funzioni indipendenti $h_i(x)$ con $1 \leq i \leq n$. Tali hash sono usati come indici in n array distinti $A_1, \ldots, A_n$ di m elementi ciascuno. Ad ogni pacchetto arrivato si incrementa la $h_i(x)$-esima posizione di ognuno del i-esimo array per tutti gli i.

Se $A_i[h_i(x)] \geq S$ per ogni i, l'indentificativo x viene salvato su disco.

Tutti gli array sono azzerati ogni $T = 1\,\text{s}$.

(a) Considerando un link bidirezionale con capacità $C = 1\,\text{Gbit/s}$, dire quanti bit occorrono per rappresentare gli n array.

> **Soluzione:** Nel caso peggiore, in un secondo si accumulano 10^9 bit in un unico contatore. Contando in ottetti, servono
>
> $$\lceil \log_2 10^9/8 \rceil mn = 27mn \text{ bit}$$

(b) Dire quali campi occorre scegliere per rappresentare l'identificativo del flusso per avere nello stesso flusso tutti i pacchetti della stessa connessione TCP.

> **Soluzione:**
>
> $$x = \text{IPs} \| \text{IPd} \| \text{proto} \| \text{ports} \| \text{portd}$$

(c) Dire come scegliere la soglia S in modo che si scriva su disco l'identificativo di tutti i flussi di almeno 25 Mbit/s. Ci sono falsi positivi? Ci sono falsi negativi?

> **Soluzione:** La soglia va fissata a $S = 3\,125\,000$ ottetti. Ci possono essere falsi positivi, non ci sono falsi negativi.

(d) Un certo flusso y ha superato i 25 Mbit/s nell'attuale intervallo di misura. Considerando $m = 2^8$ e $n = 3$, dire qual è la probabilità che il primo pacchetto di un nuovo flusso sia copiato su disco. *Ignorare gli eventuali altri flussi nel sistema.*

> **Soluzione:** Si ha una collisione se il secondo flusso x collide con y su tutti gli hash. Quiesto accade con probabilità:
>
> $$\left(\frac{1}{m}\right)^n = \frac{1}{2^{24}} = 6 \cdot 10^{-8}$$

(e) Spiegare come si può implementare la stessa funzionalità descritta in questo esercizio ma senza falsi positivi. Qual è il prezzo da pagare? In quali casi è ragionevole usare un algoritmo esatto?

> **Soluzione:** Si può usare un algoritmo di exact match per tenere traccia del volume di traffico generato da ogni flusso. Il prezzo è un aumento del tempo di esecuzione dell'algoritmo. Si può usare se i flussi distinti sono pochi.

Esercizio 3

Si consideri un filtro di Bloom costituito da un array di 128 bit e 3 funzioni hash. Inizialmente tutti i bit dell'array sono posti a zero.

(a) Calcolare la probabilità che un dato bit sia posto ad 1 dopo aver effettuato l'inserimento di 10 elementi

Soluzione: Sia m il numero di bits del filtro, k il numero di funzioni di hash e n il numero di inserimenti. La probabilità si ottiene come:

$$1 - \left(1 - \frac{1}{m}\right)^{kn} \approx \exp(-kn/m) = \exp(-30/128) \approx 0.79$$

(b) Calcolare la probabilità di un falso positivo.

Soluzione: Usando il risultato precedente si ha:

$$(1 - \exp(-kn/m))^k \approx 0.009$$

(c) Dire se la funzione $h(x) = 4x \bmod 128$ è una buona funzione di hash per questo filtro. Spiegare perché.

Soluzione: No, perché gli ultimi due bit sono sempre zero.

(d) Sono date le funzioni hash

$$h_1(x) = 3x \bmod 128$$
$$h_2(x) = 5x \bmod 128$$
$$h_3(x) = 75x \bmod 128$$

Inizialmente tutti i bit dell'array sono posti a zero. Inserire $x = 1$ e $x = 4$ nel filtro di Bloom. Dire quali bit sono posti a 1 dopo ciascun inserimento.

Soluzione:
Dopo il primo inserimento: {3,5,75}. Dopo il secondo inserimento: {3,5,12,20,44,75}.

(e) Conclusi gli inserimenti, verificare l'appartenza al dataset degli elementi $x = 5$ e $x = 257$. Dire se ci sono falsi positivi.

Soluzione:
Per $x = 5$ i bit da verificare sono 15, 25, 119. Quindi l'esito è negativo.
Per $x = 257$ i bit da verificare sono 3, 2, 75. Quindi l'esito è un (falso) positivo.